

Applying Inverse Reinforcement Learning to understand Animal Movement

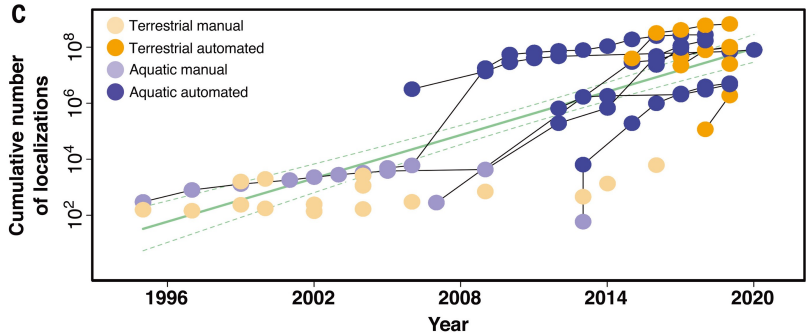
Towards IRL in Fish via Diel Vertical Migration in Cod

David R. M. Graf

September 11, 2026

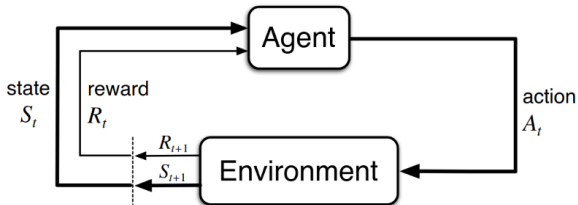
GEOMAR & CAU

Data Availability in Animal Tracking

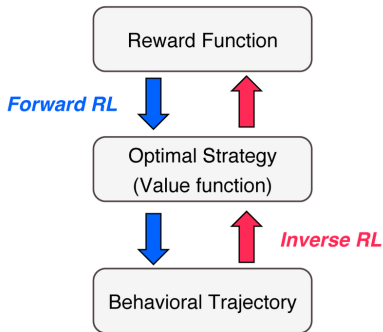


From: Nathan et al. [2022]: "Big-Data Approaches Lead to an Increased Understanding of the Ecology of Animal Movement"

Reinforcement Learning is learning what to do — how to map situations to actions — so as to maximize a numerical reward signal



From: Sutton and Barto [2020]



Inverse Reinforcement Learning is deriving a reward function from observed behavior

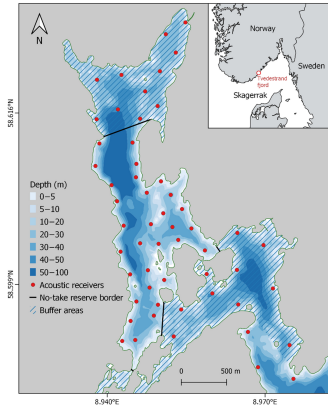
Application: DVM in Cod through MaxEnt IRL

Diel Vertical Migration in Cod

Diel Vertical Migration is defined as a
*repeated shift between shallow water occupancy during the night
and deep water occupancy during the day [Espeland et al., 2010]*

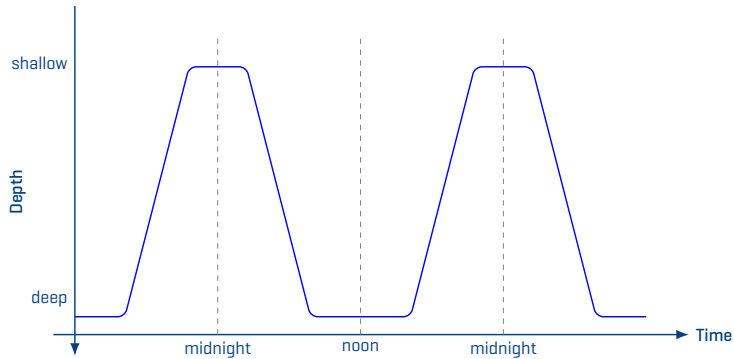
- not only in cod: well-known phenomenon in aquatic organisms [Gliwicz, 1986]
- documented across many taxa – from copepods to fish fry to adult fish of various trophic levels
- several non-exclusive explanations/trade offs:
 - food availability vs. predation risk
 - exploitation of temperature gradient – “hunt warm, rest cool”

Tvedestrand Dataset



Monk et al. [2023]

- Tvedestrand Fjord in South-Eastern Norway
- Ongoing Tagging in May/June between 2013 and 2020
- Huge Dataset: > 3, 000, 000 localisations of 600+ individuals
- Individual Cod tracked for up to 18 months
- Filter: only August, only 2014 - 2020, only trains with $Q_{.15}(\text{Depth}) - Q_{.85}(\text{Depth}) \geq 10 \text{ m}$



Application: DVM in Cod through MaxEnt IRL

Maximum Entropy IRL

Maximum Entropy IRL: Concept i

- **Goal:** Given an MDP and expert demonstrations $\mathcal{D} = \{\tau_i\}_{i=1}^n$, recover the unknown reward $\mathcal{R}$.
- **Assumption:** Reward is linear in state features ϕ : $\mathcal{R}(\tau) = \omega^T \phi(\tau)$

Feature Expectation Matching:

- We want a policy that matches the expert's behavior [even though we learn reward!]:

$$\mathbb{E}_{\pi_{\text{learned}}}[\phi(\tau)] = \mathbb{E}_{\text{expert}}[\phi(\tau)]$$

- Underspecified problem - many policies can match these expectations.

The Maximum Entropy Solution [Ziebart et al., 2008]:

- *Idea*: Resolve ambiguity by choosing the distribution with the **highest entropy** [commit to no hidden assumptions].
- *Result*: Trajectories with higher rewards become exponentially more likely:

$$p(\tau \mid \omega) \propto \exp(\omega^T \phi(\tau))$$

MaxEnt IRL Algorithm

Objective: Find weights ω that maximize the log-likelihood of the expert demonstrations.

Gradient: Difference between the expert's feature counts and our current policy's expected counts:

$$\nabla_{\omega} \mathcal{L}(\omega) = \underbrace{\mathbb{E}_{\text{expert}}[\phi(\tau)]}_{\text{What the expert does}} - \underbrace{\mathbb{E}_{\pi_{\omega}}[\phi(\tau)]}_{\text{What our policy does}}$$

Iterative Optimization Loop

1. **Solve MDP:** Compute expected state visitations D_{s_i} for the current reward ω [requires a backward value pass and forward state-visitation pass].
2. **Compute Learner Features:** $\mathbb{E}_{\pi_{\omega}}[\phi(\tau)] = \sum_{s_i} D_{s_i} \phi(s_i)$
3. **Update Weights:** If expert visits a feature more than us, increase its weight [and vice versa]: $\omega \leftarrow \omega + \eta \nabla_{\omega} \mathcal{L}(\omega)$.

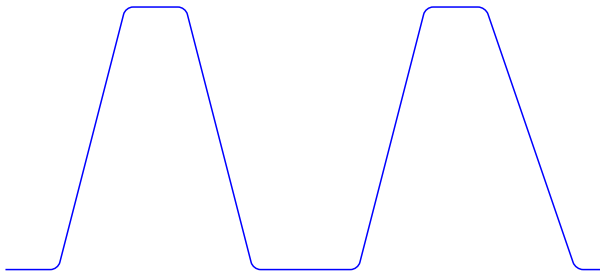
What is going on?

1. **Task:** find out how "rewarding" each feature is, based on how much the experts used it
2. **Trick 1:** any guess we pick should have a feature usage similar to the experts' [on average]
3. **Trick 2:** of all possible guesses, pick the least committed one [don't overfit!]
4. **Finally:** step-by-step, nudge your current guess towards matching the expert until the improvements get small

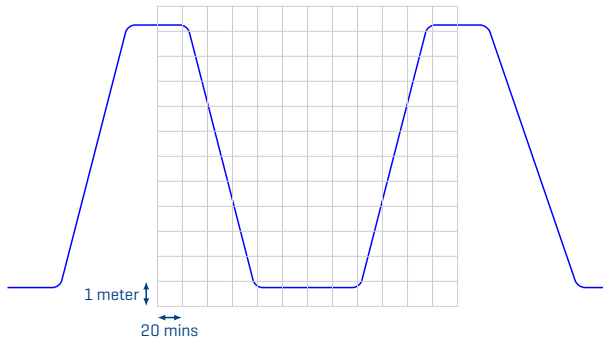
Application: DVM in Cod through MaxEnt IRL

Data Carpentry

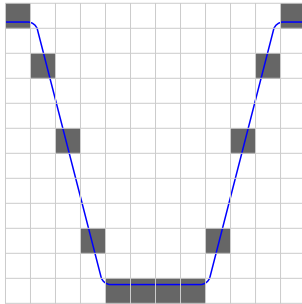
Discretization of Cod Movement



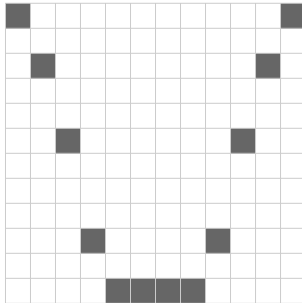
Discretization of Cod Movement



Discretization of Cod Movement

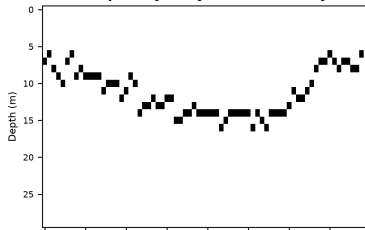


Discretization of Cod Movement

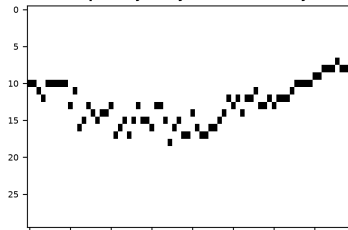


Example Trajectories in $\mathcal{S}$

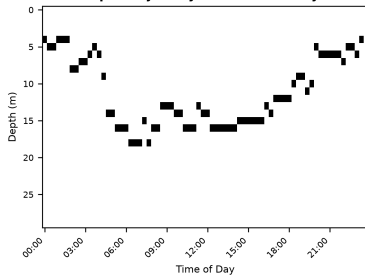
Example Trajectory — Individual-Day 9



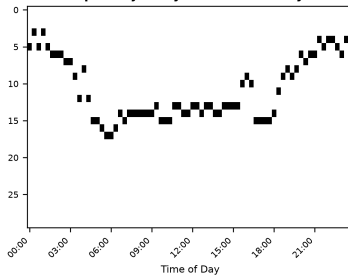
Example Trajectory — Individual-Day 16



Example Trajectory — Individual-Day 102



Example Trajectory — Individual-Day 123



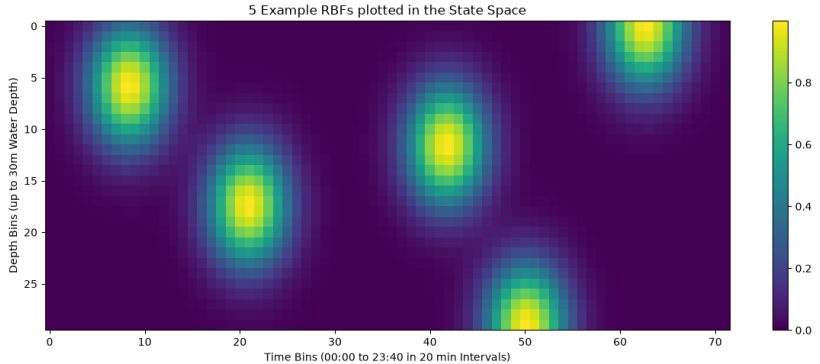
First Iteration - 1-Hot State Features

- 1-Hot Encoding
- 1 reward per state, for a total of $30 \cdot 72 = 2160$ features

Second Iteration - Radial Basis Functions

- Radial Basis Functions - "Gaussian Humps"
- One Center every 1.4h [$\sigma = 0.98h$] and every 5.8m with $\sigma = 4.1m$
- Total of $18 \cdot 6 = 108$ features

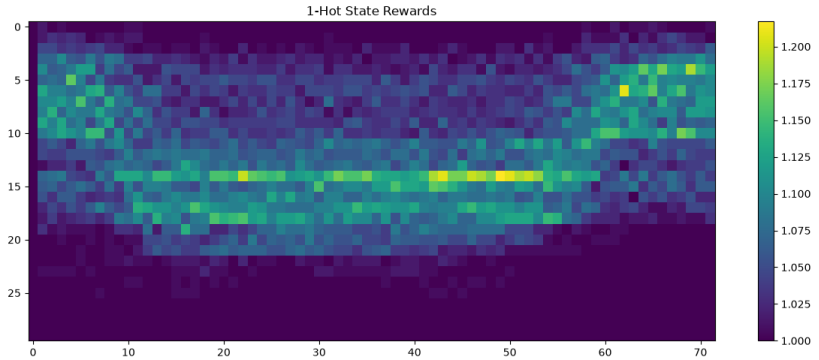
Feature Design ii



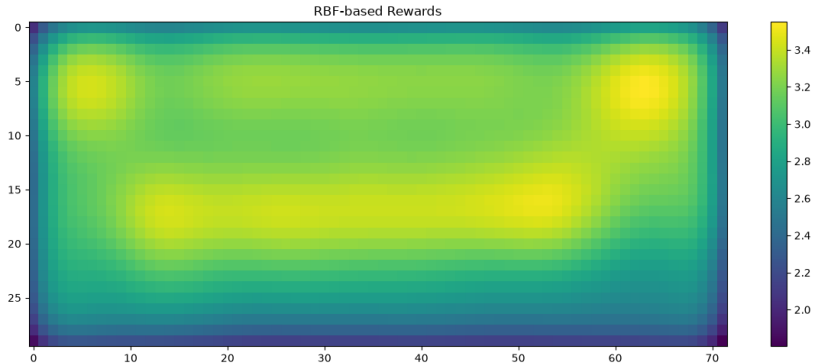
Application: DVM in Cod through MaxEnt IRL

First Results

First Iteration - 1-Hot Features



Second Iteration - RBF Features



Application: DVM in Cod through MaxEnt IRL

Now what?

- All trajectories under comparable conditions?
- Static reward map? What about switching strategies, dynamic features?
- What about movement in *all the other dimensions of space*?
- Reward only linear in the features?

Next Steps

- Provide features with more biological grounding (temperature zones, light zones, resource proxies?)
- Include movement costs
- Close the RL-Loop - simulate trajectories
- Finally: move beyond “tabular” case into free movement in 3D and dynamic features

Thanks!

Bibliography:



Get in touch!

